

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I **Vaishnavi.S (192521313)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *vaishnavi.s*

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.
4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed

messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1.A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

Reverse Engineering a Website-Scale Big Data Platform:

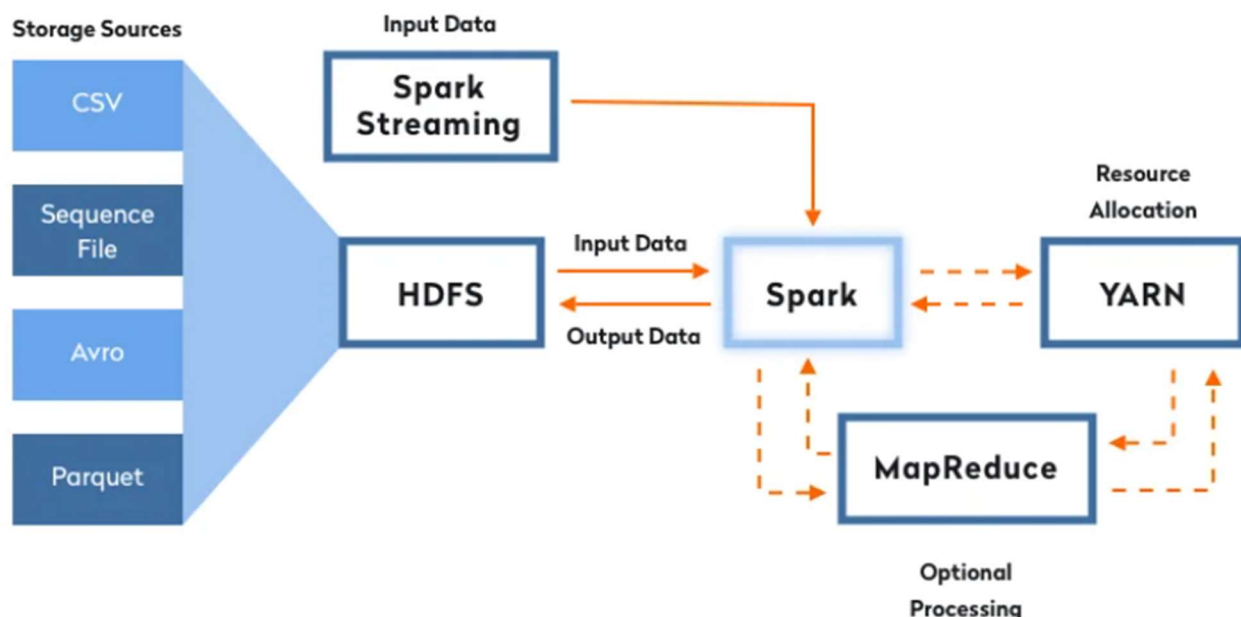
Problem Statement:

Modern websites generate massive volumes of data from user activities, searches, clicks, transactions, log files, and social interactions. Managing and analyzing this continuously generated, high-volume, and high-velocity data requires a scalable big data platform. The problem is to reverse engineer the architecture of a website-scale big data platform by identifying its data sources, ingestion mechanisms, storage systems, processing frameworks, analytics components, and visualization tools. The solution should explain how these technologies work together to support real-time and batch data processing while ensuring scalability, reliability, security, and efficient decision-making.

The storage layer is divided into :

- **Data Lake:** Stores raw clickstreams, mobile events, logs, and transaction records. Eg : HDFS , Amazon S3, Azure Lake.
- **Distributed NoSQL databases :** Handle fast reads and writes for operational analytics. Eg; Apache Cassandra or HBase
- **Data Warehouse:** Stores curated datasets for reporting. Eg : Snowflake, BigQuery, or Redshift

Probable Ingestion Framework:



Different producers continuously send:

- Website clicks
- Mobile app events
- Payment transactions
- User interactions

For scheduled imports such as CRM records or historical sales, Apache NiFi or Apache Airflow would likely manage batch ingestion.

Batch Processing Tools

The strongest candidate is Apache Spark.

Spark can:

- Aggregate billions of events
- Build customer profiles
- Generate daily reports
- Train machine learning models

Stream Processing Engine

Real-time reporting requires continuous processing.

Likely technologies include:

- Apache Spark Streaming
- Apache Flink

Flink is particularly suitable when very low latency is required because it processes events continuously rather than waiting for micro-batches.

Scalability and Partitioning Strategy

The platform probably scales horizontally.

Key strategies include:

- Kafka topic partitioning
- Spark distributed partitions
- Data partitioning by date, region, or customer ID
- Automatic cluster scaling using Kubernetes

Instead of upgrading one server, additional nodes can simply join the cluster.

Dashboard and BI Integration

Business users rarely access raw data directly.

Instead:

1. Spark writes aggregated datasets.
2. The warehouse stores optimized tables.
3. Tableau or Power BI queries those tables.
4. Executives view KPIs through live dashboards.

Examples include:

- Daily revenue
- Active users
- Conversion rate
- Regional sales

This architecture supports both historical reporting and real-time operational decisions while remaining highly scalable.

2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

Reverse Engineering an E-Commerce Personalization Platform

Problem Statement:

E-commerce platforms handle huge amounts of customer data such as browsing history, search queries, product views, purchases, ratings, and cart activity. The challenge is to design and reverse engineer a **big data personalization platform** that can collect, process, and analyze this data to understand customer preferences. The system should provide personalized product recommendations, targeted offers, and relevant search results in real time. It must also support large-scale data processing while ensuring scalability, accuracy, security, and customer privacy.

Probable Event Streaming Architecture:

The event pipeline begins with Apache Kafka. Separate topics would handle:

- Product views
- Cart updates

- Checkout events
- Payments
- Search activity

Kafka's partitioned design allows millions of simultaneous events.

Synchronizing Session, Clickstream, and Transactions

The platform probably creates a unique customer identifier.

- Session ID
- Customer ID
- Timestamp

Streaming processors merge these events into one customer timeline.

For example: A customer views shoes, adds them to the cart, abandons checkout, and later returns. The recommendation engine immediately recognizes the previous activity.

Feature Engineering

Customer behavior features include:

- Purchase frequency
- Product categories
- Time spent browsing

Ranking Model

Likely algorithms include:

- Collaborative filtering
- Gradient boosting
- Deep learning ranking models

The ranking stage produces personalized product lists.

Caching System

Real-time recommendations cannot repeatedly query large databases.

The platform would likely use Redis.

Redis stores:

- Active sessions
- Popular products
- Recent recommendations

- **Fault Tolerance**

If one server fails, another continues processing the architecture probably includes:

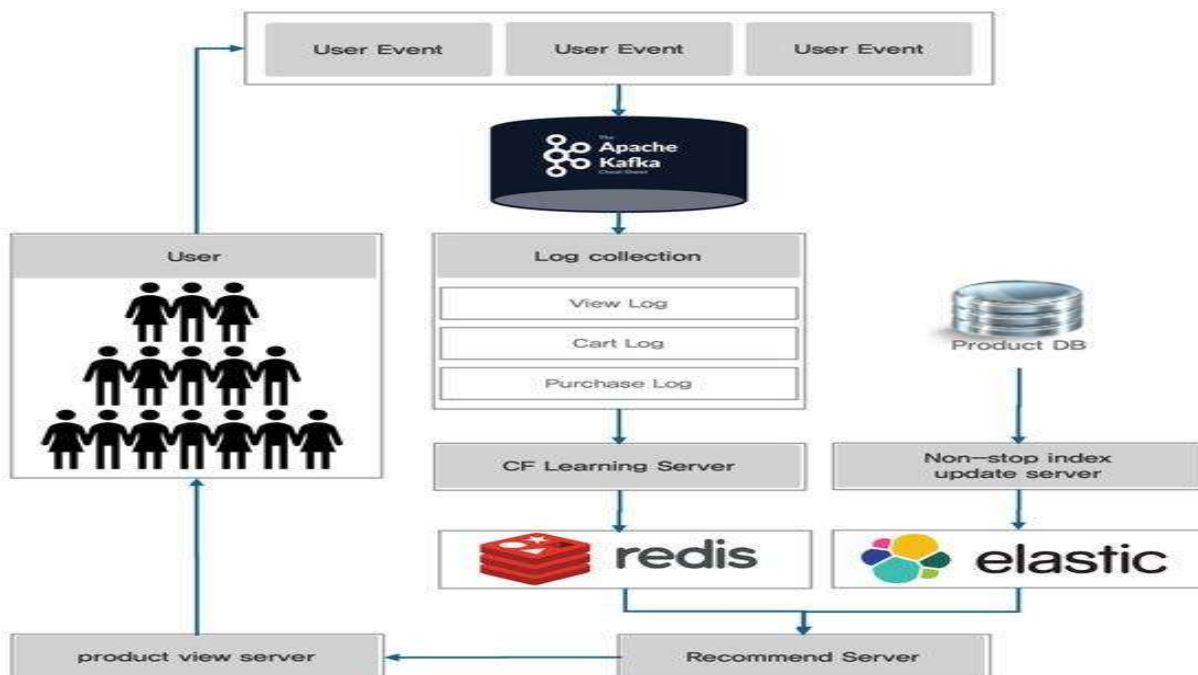
- Kafka replication
- Spark checkpointing
- Multi-region backups
- Automatic node replacement

.Dashboard Generation

Business dashboards likely display:

- Cart abandonment rate
- Conversion rate
- Revenue by region
- Recommendation performance

Power BI or Tableau would query warehouse tables refreshed continuously through the streaming pipeline.



3.A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

Reverse Engineering a Healthcare Big Data Platform

Problem Analysis

A healthcare big data platform must manage huge amounts of data generated from electronic health records, medical reports, diagnostic devices, wearable sensors, and hospital systems. This data is available in different formats and is continuously generated, making storage, integration, and processing challenging. The platform must support real-time analysis for patient monitoring and early disease prediction while maintaining high accuracy and reliability. It should also provide secure access to sensitive patient information and protect data privacy. Therefore, a scalable and secure big data platform is required to efficiently process healthcare data and support better medical decision-making.

Data Integration

Hospitals rarely use identical software. The platform therefore likely relies on interoperability standards. These standards allow hospitals, laboratories, and insurers to exchange information consistently.

- HL7
- FHIR APIs
- DICOM

Streaming Framework

Apache Kafka probably receives these events. Apache Spark Streaming or Flink analyzes them immediately. Wearable devices continuously generate:

- Heart rate
- Blood pressure
- Oxygen levels

Machine Learning Framework

Healthcare prediction likely involves:

- TensorFlow
- PyTorch
- Spark MLlib

Possible models include:

- Disease risk prediction
- Readmission prediction

- Early warning detection

Example:

An abnormal heart-rate pattern triggers an alert before symptoms worsen.

Privacy and Compliance

Compliance would likely align with regulations such as HIPAA or national healthcare privacy laws. Healthcare platforms require extremely strong security.

- AES-256 encryption
- TLS transmission
- Role-based access control
- Multi-factor authentication
- Audit logging

Analytics Pipeline

The pipeline probably follows this sequence:

1. Collect hospital data.
2. Integrate through FHIR.
3. Stream wearable data.
4. Store raw data.
5. Process using Spark.
6. Generate predictions.
7. Display clinician dashboards.

Population health studies would analyze aggregated, de-identified records rather than exposing individual patient information.

4.A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

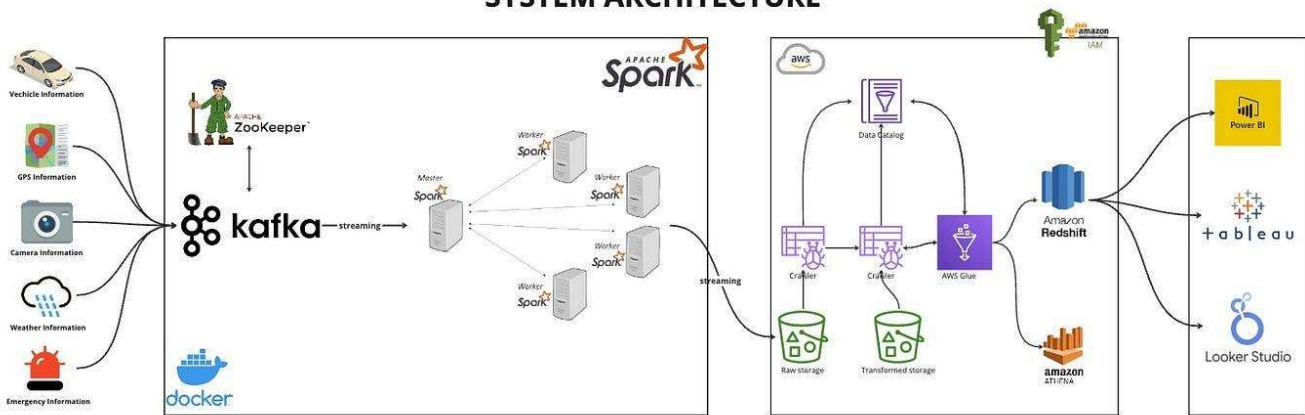
Reverse Engineering a Smart City Platform

Problem Analysis

A smart city continuously receives traffic sensor data, CCTV metadata, weather updates, and public transport activity. Because thousands of devices generate information simultaneously, much of the processing likely begins near the data source before reaching centralized systems.

Probable Edge Computing Setup

SYSTEM ARCHITECTURE



- SENSOR MOUNTING
- Local event detection

Instead of transmitting entire video streams, cameras may send metadata describing detected vehicles. This reduces bandwidth significantly.

Distributed Messaging System

Apache Kafka is the strongest candidate.

Different topics likely exist for:

- Traffic sensors
- Weather stations
- Public transport
- CCTV metadata

Kafka allows independent processing of each data stream.

Geospatial Analytics

Traffic prediction requires location-aware processing.

Likely technologies include:

- Apache Sedona
- PostGIS These systems perform operations such as:
- Route calculation
- Congestion mapping
- Geographic clustering

Streaming Computation

Apache Flink is particularly suitable because congestion prediction requires continuous event processing.

Example:

A sudden increase in vehicle counts automatically updates traffic predictions.

Time-Series Database

Traffic measurements naturally fit time-series storage.

Likely databases include:

- InfluxDB
- TimescaleDB

These efficiently store timestamped sensor readings.

Combining Batch and Real-Time Analytics

Two analytical paths likely operate simultaneously.

Real-time

- Live congestion
- Incident detection
- Signal adjustments

Batch

- Monthly traffic studies
- Infrastructure planning
- Seasonal analysis

Together they support immediate operations and long-term urban planning.

Security Measures

Because city infrastructure is sensitive, protections likely include:

- Device authentication
- Encrypted communication
- Network segmentation
- Role-based permissions
- Continuous monitoring

Visualization

Operations centers likely use GIS dashboards showing:

- Live traffic
- Bus movement
- Weather alerts
- Road incidents

These dashboards update continuously from streaming analytics.

5.A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering a Healthcare Analytics Platform

Problem Analysis

The healthcare system collects large volumes of data from patient records, wearable devices, laboratory reports, and appointment histories. The main challenge is to securely integrate structured and unstructured medical data and process both historical and real-time information for effective healthcare analytics.

Probable Hybrid Storage Architecture

- Relational Database: Patient records, appointments, billing, and other structured data.
- Data Lake/Object Storage: Medical images, reports, documents, and sensor data.
- Data Warehouse/Lakehouse: Integrated and cleaned data for analytics and reporting.
- Backup Storage: Secure backup systems for disaster recovery.
- Design Decision: Hybrid storage provides flexibility, scalability, and secure handling of different data types.

ETL Pipeline and Interoperability

- Extract: Data collected from hospitals, clinics, laboratories, and wearable devices.
- Transform: Data is cleaned, validated, standardized, and converted into compatible formats.
- Load: Processed data is stored in databases, data lakes, or warehouses.
- Interoperability Standards: HL7 and FHIR can be used to exchange healthcare information between different systems.
- APIs: Enable secure communication between hospitals, clinics, and healthcare applications.

Real-Time Monitoring and Predictive Analytics

- Wearable devices continuously generate patient health readings.
- Streaming systems process incoming data in real time.
- Abnormal readings can generate alerts for healthcare professionals.
- Historical patient data can be combined with real-time data.

- Predictive analytics can identify possible health risks and support early intervention.

Distributed Processing

- Frameworks such as Apache Spark can process massive healthcare datasets.
- Data is divided across multiple computing nodes and processed in parallel.
- Supports population-level analysis of millions of patient records.
- Helps identify disease trends, risk patterns, and treatment outcomes.
- Provides faster processing and better scalability.

Security, Compliance and Audit

- Encryption: Protects patient data during storage and transmission.
- Authentication: Verifies users before providing access.
- Role-Based Access Control: Limits access according to user responsibilities.
- Audit Logs: Record data access, modification, and other activities.
- Data Privacy: Sensitive patient information must be protected from unauthorized access.
- The platform should comply with applicable healthcare data-protection and privacy regulations.

Machine Learning Applications

- Diagnosis Risk Scoring: Predicts the probability of diseases based on patient data.
- Early Risk Detection: Identifies patients who may develop health complications.
- Treatment Support: Analyzes previous cases and patient characteristics to assist treatment decisions.
- Patient Monitoring: Detects unusual patterns from wearable-device data.
- Outcome Prediction: Predicts possible treatment outcomes and patient risks.
- ML outputs should support healthcare professionals rather than replace their clinical judgment.

